

Nolyth — Official Sprint Guide Sprint 03

Deep Learning, NLP & Transformers

Neural Network Foundations

Good — here's the same concept, but grounded in a concrete example instead of pure formula, since that's usually what makes it click.

Quick term glossary

Term	Plain meaning	Real-world analogy
Neuron	A number between 0-1 (its "activation") — how strongly that feature is present	A dimmer switch — off (0) to fully on (1)
Weight	How much one neuron's output influences the next neuron	Volume knob on each input wire
Bias	A number added before deciding "on or off" — shifts the threshold	How stubborn the switch is to turn on
Activation function	Squashes the weighted sum into a usable range (e.g. 0-1)	The "decision" step — did enough signal arrive to fire?

The 4-layer walkthrough (3Blue1Brown's own digit example)

Think of recognizing a handwritten digit (0-9):

Layer 1 — Input (784 neurons) Each neuron = one pixel's brightness (0 = black, 1 = white). No weights or activation here — this is just the raw picture, unrolled into numbers.

Layer 2 — Hidden layer "Edges" (say, 16 neurons) Each neuron here should light up when it detects a small edge or line segment somewhere in the image — e.g. "is there a short vertical stroke in the top-left?"

→ **This is exactly where weights first appear.** Every one of the 784 input neurons connects to every one of these 16 neurons, each connection has its own weight (784 ×

16 weights just for this layer). The neuron sums up: $(\text{pixel}_1 \times \text{weight}_1) + (\text{pixel}_2 \times \text{weight}_2) + \dots + \text{bias}$.

→ **Activation function applied here too.** That raw sum could be any huge number (positive or negative). Sigmoid squashes it into 0-1, so the neuron cleanly says "edge present" (~ 1) or "edge absent" (~ 0).

Layer 3 — Hidden layer "Patterns/Loops" (say, 16 neurons) Each neuron combines several "edge" detections from Layer 2 into a bigger shape — e.g. "do these edges form a closed loop?" (useful for detecting 0, 6, 8, 9).

Same story: weights connect every Layer-2 neuron to every Layer-3 neuron, sum + bias, then squashed by activation.

Layer 4 — Output (10 neurons, one per digit 0-9) Combines the "loop/pattern" detections into a final vote: "how much does this look like a 9?" The neuron with the highest activation is the model's guess.

Little math comparison (from your image, but line-by-line)

For one neuron in Layer 2:

$$\text{raw_sum} = (w_0 \times \text{pixel}_0) + (w_1 \times \text{pixel}_1) + \dots + (w_n \times \text{pixel}_n) + \text{bias}$$

$$\text{activation} = \text{sigmoid}(\text{raw_sum})$$

- **Weights** live on every arrow between layers — that's what gets multiplied.
- **Bias** is added once per neuron, after the weighted sum, before squashing.
- **Activation** is applied last, per neuron, to produce the final 0-1 value that gets passed to the next layer.

Why bias, specifically?

Without bias, a neuron can only fire when the weighted sum crosses **zero**. But maybe you want a neuron to only activate when the evidence is *strongly* positive (say, $\text{sum} > 5$), or to be a bit trigger-happy and fire even with weak evidence ($\text{sum} > -3$). Bias shifts that threshold — it's the neuron's personal "how convinced do I need to be" setting, independent of the inputs themselves.



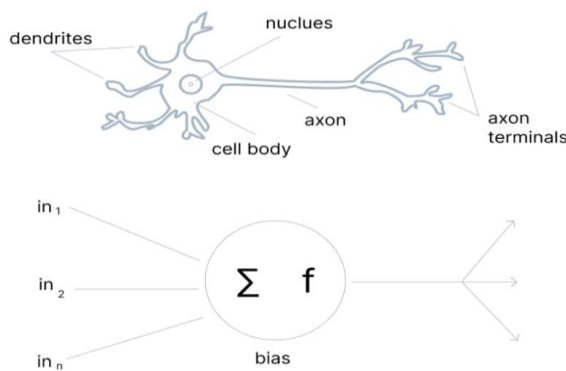
Activation Functions

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.

The role of the Activation Function is to derive output from a set of input values fed to a node (or a layer).

But - Let's take a step back and clarify: What exactly is a node?

Well, if we compare the neural network to our brain, a node is a replica of a neuron that receives a set of input signals—external stimuli.



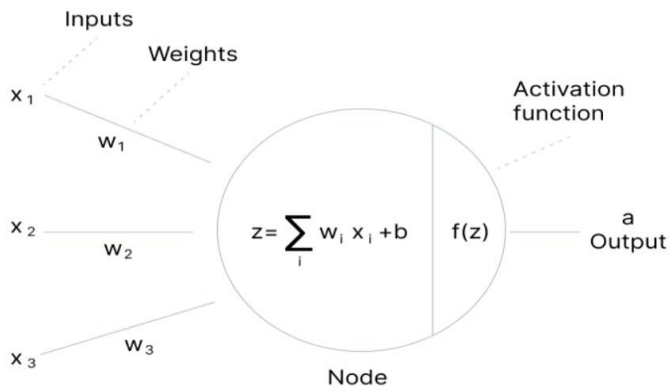
V7 Labs

Depending on the nature and intensity of these input signals, the brain processes them and decides whether the neuron should be activated ("fired") or not.

Depending on the nature and intensity of these input signals, the brain processes them and decides whether the neuron should be activated ("fired") or not.

In [deep learning](#), this is also the role of the Activation Function—that's why it's often referred to as a Transfer Function in Artificial Neural Network.

The primary role of the Activation Function is to transform the summed weighted input from the node into an output value to be fed to the next hidden layer or as output.



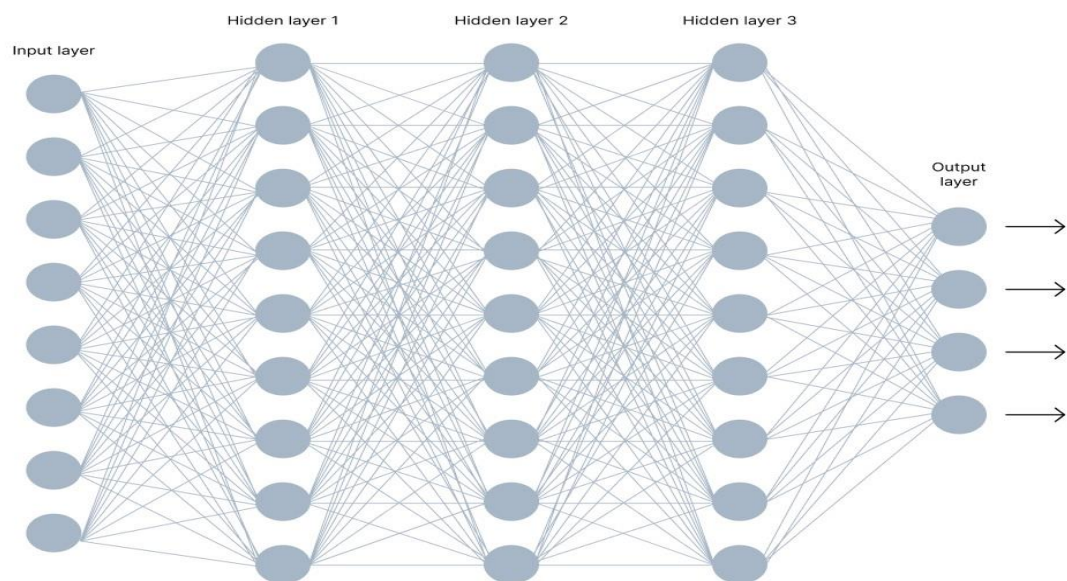
V7 Labs

Now, let's have a look at the Neural Networks Architecture.

[Elements of a Neural Networks Architecture](#)

Here's the thing - If you don't understand the concept of neural networks and how they work, diving deeper into the topic of activation functions might be challenging.

That's why it's a good idea to refresh your knowledge and take a quick look at the structure of the Neural Networks Architecture and its components. Here it is.



V7 Labs

In the image above, you can see a neural network made of interconnected neurons. Each of them is characterized by its weight, bias, and activation function.

Here are other elements of this network.

Input Layer

The input layer takes raw input from the domain. No computation is performed at this layer. Nodes here just pass on the information (features) to the hidden layer.

Hidden Layer

As the name suggests, the nodes of this layer are not exposed. They provide an abstraction to the neural network.

The hidden layer performs all kinds of computation on the features entered through the input layer and transfers the result to the output layer.

Output Layer

It's the final layer of the network that brings the information learned through the hidden layer and delivers the final value as a result.

All hidden layers usually use the same activation function. However, the output layer will typically use a different activation function from the hidden layers. The choice depends on the goal or type of prediction made by the model.

Types of Activation Functions

1. **ReLU (Rectified Linear Unit):** Used in hidden layers to introduce non-linearity while keeping computation efficient.
2. **Sigmoid:** Used in binary classification problems.
3. **Softmax:** Used for multi-class classification.
4. **Tanh:** Similar to sigmoid but with better gradient properties.

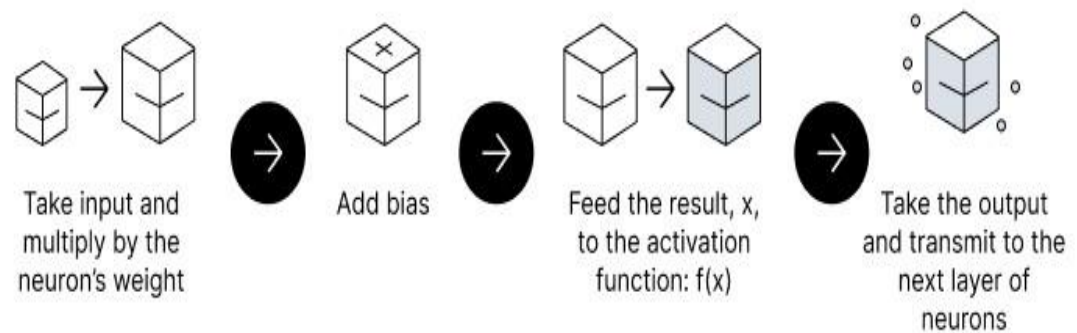
Feedforward vs. Backpropagation

When learning about neural networks, you will come across two essential terms describing the movement of information—feedforward and backpropagation.

Let's explore them.

Feedforward Propagation - the flow of information occurs in the forward direction. The input is used to calculate some intermediate function in the hidden layer, which is then used to calculate an output.

In the feedforward propagation, the Activation Function is a mathematical “gate” in between the input feeding the current neuron and its output going to the next layer.



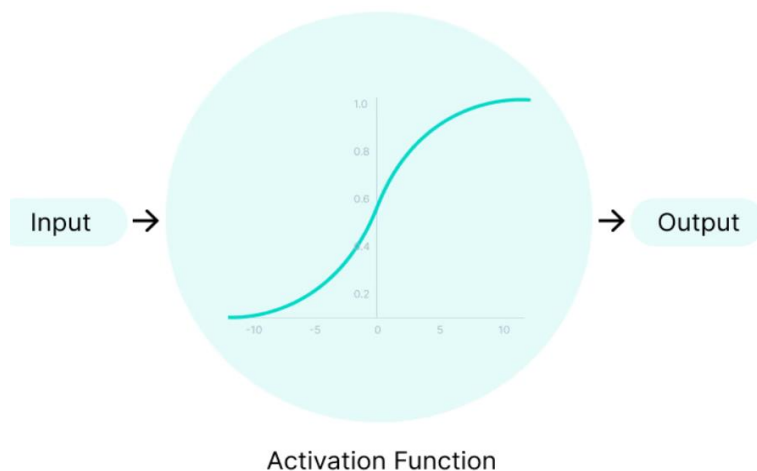
V7 Labs

Backpropagation - the weights of the network connections are repeatedly adjusted to minimize the difference between the actual output vector of the net and the desired output vector.

To put it simply—backpropagation aims to minimize the cost function by adjusting the network’s weights and biases. The cost function gradients determine the level of adjustment with respect to parameters like activation function, weights, bias, etc.

Why do Neural Networks Need an Activation Function?

So, we know what Activation Function is and what it does, but -Why do Neural Networks need it? Well, the purpose of an activation function is to add non-linearity to the neural network.



Activation functions introduce an additional step at each layer during the forward propagation, but its computation is worth it. Here is why—

Let's suppose we have a neural network working without the activation functions.

In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases. It's because it doesn't matter how many hidden layers we attach in the neural network; all layers will behave in the same way because the composition of two linear functions is a linear function itself.

Although the neural network becomes simpler, learning any complex task is impossible, and our model would be just a linear regression model.

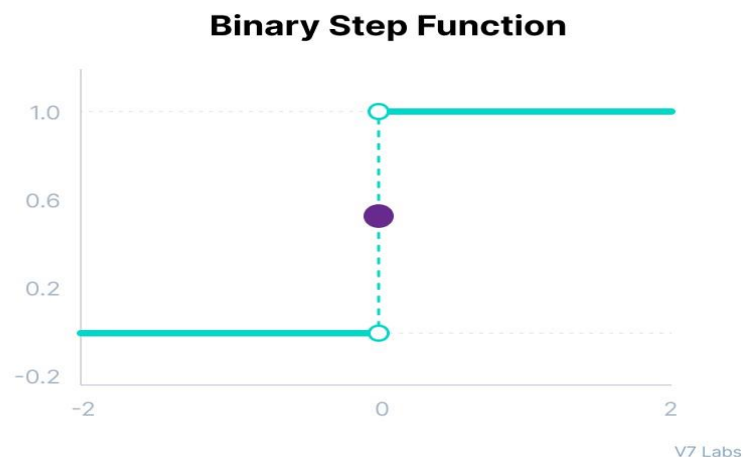
3 Types of Neural Networks Activation Functions

Now, as we've covered the essential concepts, let's go over the most popular neural networks activation functions.

Binary Step Function

Binary step function depends on a threshold value that decides whether a neuron should be activated or not.

The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.



Mathematically it can be represented as:

Binary step

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

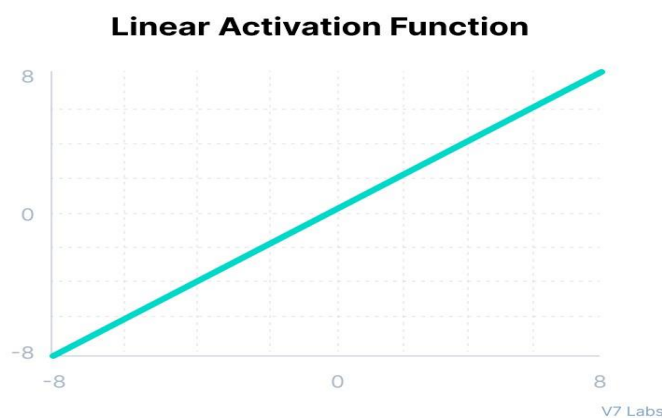
Here are some of the limitations of binary step function:

- It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
- The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

Linear Activation Function

The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the activation is proportional to the input.

The function doesn't do anything to the weighted sum of the input; it simply spits out the value it was given.



Mathematically it can be represented as:

Linear

$$f(x) = x$$

However, a linear activation function has two major problems:

- It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x .
- All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

Non-Linear Activation Functions

The linear activation function shown above is simply a linear regression model.

Because of its limited power, this does not allow the model to create complex mappings between the network's inputs and outputs.

Non-linear activation functions solve the following limitations of linear activation functions:

- They allow backpropagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

Now, let's have a look at ten different non-linear neural networks activation functions and their characteristics.

10 Non-Linear Neural Networks Activation Functions

Sigmoid / Logistic Activation Function

This function takes any real value as input and outputs values in the range of 0 to 1.

The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0, as shown below.



Mathematically it can be represented as:

Sigmoid / Logistic

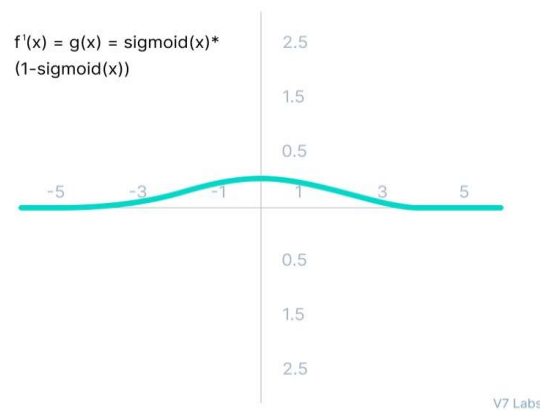
$$f(x) = \frac{1}{1 + e^{-x}}$$

Here's why sigmoid/logistic activation function is one of the most widely used functions:

- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

The limitations of sigmoid function are discussed below:

- The derivative of the function is $f'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$.



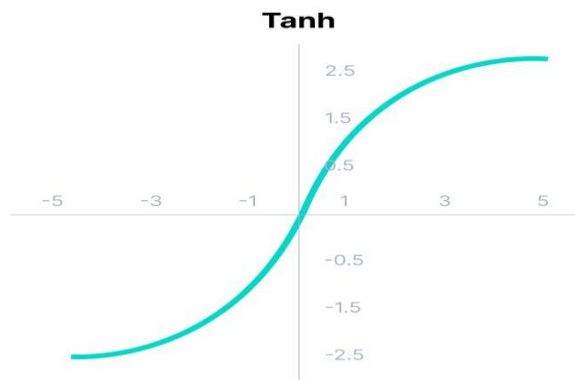
As we can see from the above Figure, the gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.

It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the Vanishing gradient problem.

- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the [training of the neural network](#) more difficult and unstable.

Tanh Function (Hyperbolic Tangent)

Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1. In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.



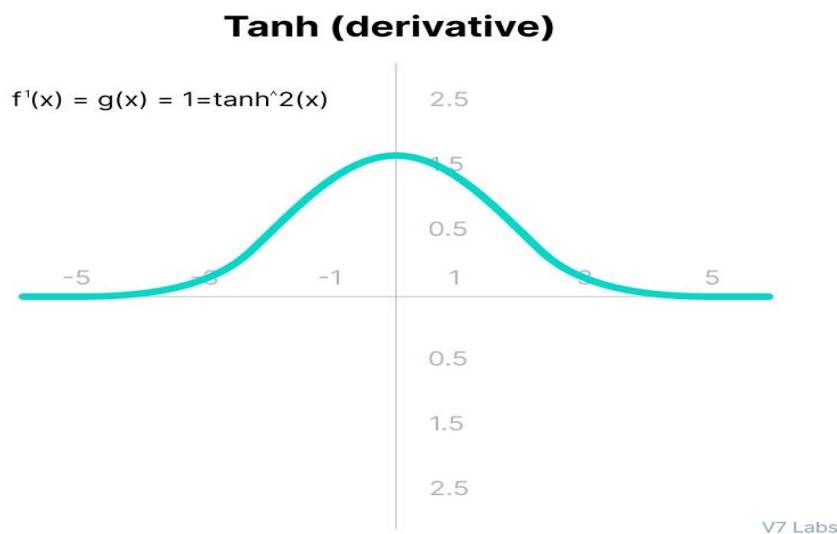
Mathematically it can be represented as:

$$\text{Tanh}$$
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

Advantages of using this activation function are:

- The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- Usually used in hidden layers of a neural network as its values lie between -1 to; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in centering the data and makes learning for the next layer much easier.

Have a look at the gradient of the tanh activation function to understand its limitations.



As you can see— it also faces the problem of vanishing gradients similar to the sigmoid activation function. Plus the gradient of the tanh function is much steeper as compared to the sigmoid function.

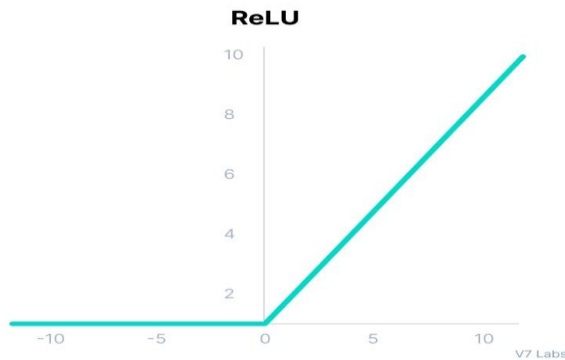
Note: Although both sigmoid and tanh face vanishing gradient issue, tanh is zero centered, and the gradients are not restricted to move in a certain direction. Therefore, in practice, tanh nonlinearity is always preferred to sigmoid nonlinearity.

ReLU Function

ReLU stands for Rectified Linear Unit. Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.

The main catch here is that the ReLU function does not activate all the neurons at the same time.

The neurons will only be deactivated if the output of the linear transformation is less than 0.



ReLU

$$f(x) = \max(0, x)$$

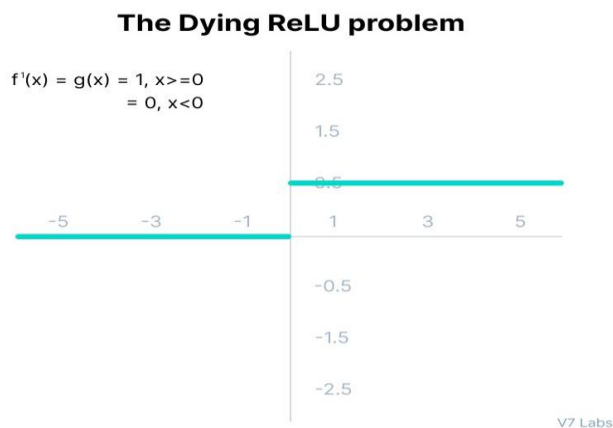
Mathematically it can be represented as:

The advantages of using ReLU as an activation function are as follows:

- Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.
- ReLU accelerates the convergence of gradient descent towards the global minimum of the [loss function](#) due to its linear, non-saturating property.

The limitations faced by this function are:

- The Dying ReLU problem, which I explained below.



The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can create dead neurons which never get activated.

- All the negative input values become zero immediately, which decreases the model's ability to fit or train from the data properly.

Note: For building the most reliable ML models, split your data into train, validation, and test sets.

[Leaky ReLU Function](#)

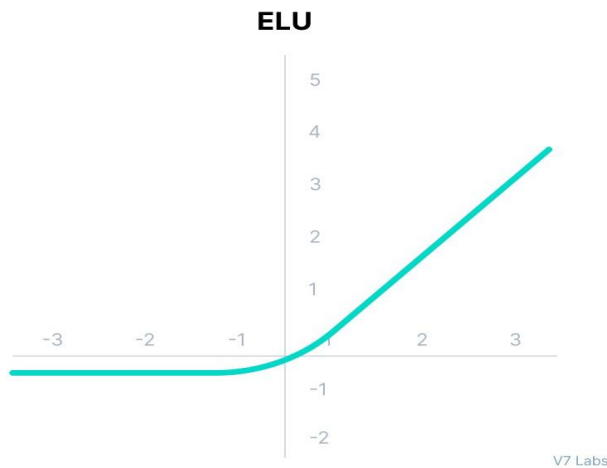
[Parametric ReLU Function](#)

[Exponential Linear Units \(ELUs\) Function](#)

[Exponential Linear Units \(ELUs\) Function](#)

Exponential Linear Unit, or ELU for short, is also a variant of ReLU that modifies the slope of the negative part of the function.

ELU uses a log curve to define the negative values unlike the leaky ReLU and Parametric ReLU functions with a straight line.



Mathematically it can be represented as:

ELU

$$\begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$$

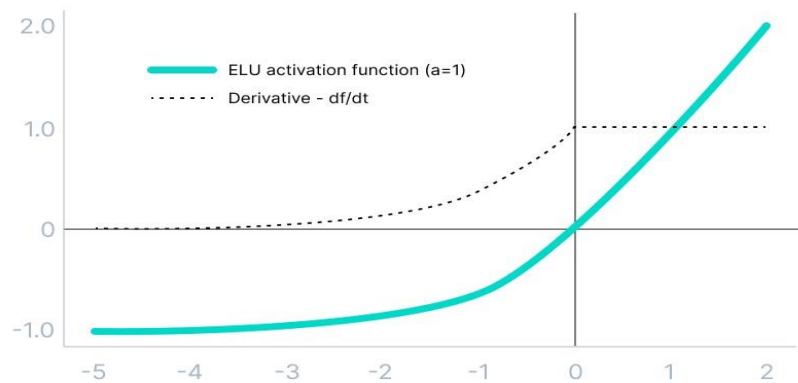
ELU is a strong alternative for f ReLU because of the following advantages:

- ELU becomes smooth slowly until its output equal to $-\alpha$ whereas ReLU sharply smooths.
- Avoids dead ReLU problem by introducing log curve for negative values of input. It helps the network nudge weights and biases in the right direction.

The limitations of the ELU function are as follow:

- It increases the computational time because of the exponential operation included
- No learning of the 'a' value takes place
- Exploding gradient problem

ELU (a=1) + Derivative



V7 Labs

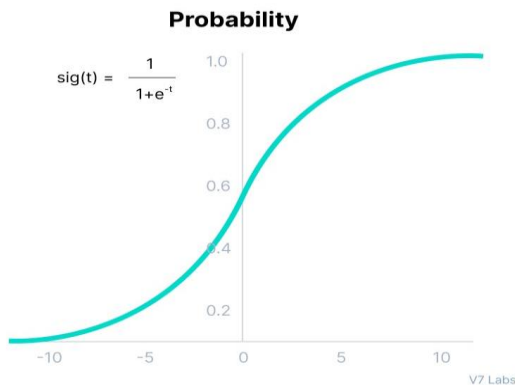
Mathematically it can be represented as:

ELU derivative

$$f'(x) = \begin{cases} 1 & \text{for } x \geq 0 \\ f(x) + \alpha & \text{for } x < 0 \end{cases}$$

SoftMax Function

Before exploring the ins and outs of the Softmax activation function, we should focus on its building block—the sigmoid/logistic activation function that works on calculating probability values.



The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.

But— This function faces certain problems.

Let's suppose we have five output values of 0.8, 0.9, 0.7, 0.8, and 0.6, respectively. How can we move forward with it?

The answer is: We can't. The above values don't make sense as the sum of all the classes/output probabilities should be equal to 1.

You see, the Softmax function is described as a combination of multiple sigmoids. It calculates the relative probabilities. Similar to the sigmoid/logistic activation function, the SoftMax function returns the probability of each class.

It is most commonly used as an activation function for the last layer of the neural network in the case of multi-class classification. Mathematically it can be represented as:

Softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Let's go over a simple example together.

Assume that you have three classes, meaning that there would be three neurons in the output layer. Now, suppose that your output from the neurons is [1.8, 0.9, 0.68].

Applying the softmax function over these values to give a probabilistic view will result in the following outcome: [0.58, 0.23, 0.19].

The function returns 1 for the largest probability index while it returns 0 for the other two array indexes. Here, giving full weight to index 0 and no weight to index 1 and index 2. So the output would be the class corresponding to the 1st neuron(index 0) out of three.

You can see now how softmax activation function make things easy for multi-class classification problems.

[Swish](#)

[Gaussian Error Linear Unit \(GELU\)](#)

[Scaled Exponential Linear Unit \(SELU\)](#)

[Why are deep neural networks hard to train?](#)

There are two challenges you might encounter when training your deep neural networks.

Let's discuss them in more detail.

Vanishing Gradients

Like the sigmoid function, certain activation functions squish an ample input space into a small output space between 0 and 1.

Therefore, a large change in the input of the sigmoid function will cause a small change in the output. Hence, the derivative becomes small. For shallow networks with only a few layers that use these activations, this isn't a big problem.

However, when more layers are used, it can cause the gradient to be too small for training to work effectively.

Exploding Gradients

Exploding gradients are problems where significant error gradients accumulate and result in very large updates to neural network model weights during training.

An unstable network can result when there are exploding gradients, and the learning cannot be completed.

The values of the weights can also become so large as to overflow and result in something called NaN values.

How to choose the right Activation Function?

You need to match your activation function for your output layer based on the type of prediction problem that you are solving—specifically, the type of predicted variable.

Here's what you should keep in mind.

As a rule of thumb, you can begin with using the ReLU activation function and then move over to other activation functions if ReLU doesn't provide optimum results.

And here are a few other guidelines to help you out.

1. ReLU activation function should only be used in the hidden layers.
2. Sigmoid/Logistic and Tanh functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
3. Swish function is used in neural networks having a depth greater than 40 layers.

Finally, a few rules for choosing the activation function for your output layer based on the type of prediction problem that you are solving:

1. Regression - Linear Activation Function
2. Binary Classification—Sigmoid/Logistic Activation Function
3. Multiclass Classification—Softmax
4. Multilabel Classification—Sigmoid

The activation function used in hidden layers is typically chosen based on the type of neural network architecture.

1. Convolutional Neural Network (CNN): ReLU activation function.
2. [Recurrent Neural Network](#): Tanh and/or Sigmoid activation function.

And hey—use this cheatsheet to consolidate all the knowledge on the Neural Network Activation Functions that you've just acquired :)

Neural Networks Activation Functions in a Nutshell

Well done!

You've made it this far ;-) Now, let's have a quick recap of everything you've learnt in this tutorial:

- Activation Functions are used to introduce non-linearity in the network.
- A neural network will almost always have the same activation function in all hidden layers. This activation function should be differentiable so that the parameters of the network are learned in backpropagation.
- ReLU is the most commonly used activation function for hidden layers.
- While selecting an activation function, you must consider the problems it might face: vanishing and exploding gradients.
- Regarding the output layer, we must always consider the expected value range of the predictions. If it can be any numeric value (as in case of the regression problem) you can use the linear activation function or ReLU.
- Use Softmax or Sigmoid function for the classification problems.

Following the roadmap order (Days 1–2 = Neural Network Foundations), here are the first 5 topics to touch — in sequence, not skipping ahead:

1. Neurons, layers, weights, and bias

- Video: 3Blue1Brown — "But what is a neural network?" — <https://www.youtube.com/watch?v=aircAruvnKk>
- Read: <https://www.deeplearningbook.org/contents/mlp.html> (Ch. 6 intro)
- Exercise: build a single neuron from scratch in NumPy — <https://www.kaggle.com/code/prashant111/neural-network-from-scratch-in-python>

2. Activation functions and why they're needed

- Read: <https://www.v7labs.com/blog/neural-networks-activation-functions>
- Interactive: <https://playground.tensorflow.org/> (toggle activation functions live)
- Exercise: plot ReLU, Sigmoid, Tanh curves in Python/Matplotlib yourself

3. Forward propagation and model predictions

- Read: <https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html>
- Video: Andrew Ng — Neural Networks course (Course 1, Week 3) — <https://www.coursera.org/learn/neural-networks-deep-learning>
- Exercise: implement forward pass manually for a 2-layer network

4. Loss and backpropagation (conceptual level)

- Video: 3Blue1Brown — "Backpropagation, intuitively" — <https://www.youtube.com/watch?v=Ilg3gGewQ5U>
- Read: <https://ml-cheatsheet.readthedocs.io/en/latest/backpropagation.html>
- Exercise: PyTorch autograd demo — https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html

5. Training, validation, and basic PyTorch/TensorFlow workflow

- Official tutorial: <https://pytorch.org/tutorials/beginner/basics/intro.html> (PyTorch's own "Learn the Basics" series — covers tensors → training loop end to end)
- Alt (Keras): <https://www.tensorflow.org/tutorials/keras/classification>
- Exercise: train a basic classifier on MNIST using either framework, track train vs val loss

Once you're through these five (Days 1–2 territory), we move to CNNs next — not before.

NN-Frameworks

Frameworks: PyTorch vs TensorFlow/Keras — history & comparison

TensorFlow (Google Brain, released Nov 2015)

- First major deep learning framework to get wide industry adoption.
- Used a static computation graph (define-then-run): you had to build the entire graph first, then run it in a separate "session" — made debugging hard, since you couldn't inspect values mid-computation.
- Strong for production deployment from the start — TensorFlow Serving, TensorFlow Lite (mobile), TensorFlow.js (browser).

PyTorch (Meta/Facebook AI Research, released Oct 2016)

- Built specifically to fix TensorFlow's biggest pain point.
- Uses a dynamic computation graph (define-by-run): the graph is built on the fly as code executes — so you can use normal Python debugging (print(), breakpoints) at any point in the model.
- This flexibility made it the default choice for research — most new papers publish PyTorch code first.
- Became the backbone of the modern NLP/Transformer ecosystem — Hugging Face's transformers library is PyTorch-first, which matters directly for your DistilBERT work later this sprint.

Keras (originally independent, released March 2015 by François Chollet)

- Started as a separate high-level wrapper library, designed purely for simplicity — could run on top of TensorFlow, Theano, or CNTK.
- TensorFlow absorbed Keras officially in TF 2.0 (2019) as tf.keras — because TensorFlow needed to copy the ease-of-use that was pulling developers toward PyTorch.
- Result: TensorFlow 2.x today *is* mostly Keras under the hood — model.fit(), model.compile() handle the training loop for you automatically.

Core trade-off

	PyTorch	TensorFlow/Keras
Training loop	You write it manually (forward → loss → backward → step)	Handled for you via .fit()
Debugging	Native Python debugging, line by line	Historically harder; improved a lot in TF 2.x
Best for	Research, learning internals, NLP/Transformers	Production deployment, mobile/edge, quick prototyping
Ecosystem pull	Hugging Face, most research papers	Google's production tooling (TF Serving, TF Lite)

Why I'm using PyTorch for this sprint: it forces me to write out the full Input → Model → Prediction → Loss → Learning loop by hand instead of hiding it behind .fit() — and it lines up with Hugging Face for the DistilBERT fine-tuning later in Sprint 03.

Recommendation for you specifically: PyTorch. Two reasons — it's now the dominant framework in research, papers, and Hugging Face (which you'll need in Sprint 03 for DistilBERT anyway, since Hugging Face is PyTorch-first), and its manual training loop directly reinforces the Input → Model → Prediction → Loss → Learning cycle you're trying to internalize this sprint.

ANN — Artificial Neural Network

It's the simplest, most basic type of neural network — just layers of neurons stacked one after another, each fully connected to the next (every neuron in one-layer talks to every neuron in the next).

Architecture

Input Layer → Hidden Layer(s) → Output Layer

(784 neurons) → (128 → 64) → (10 neurons)

Structure breakdown (using your MNIST model as reference):

Layer	Neurons	Role
Input	784	Raw flattened pixel values (28×28 image → 1D vector)
Hidden 1	128	Learns basic feature combinations
Hidden 2	64	Refines those into higher-level patterns
Output	10	Final class scores (digits 0-9)

Key property: Every neuron in one layer connects to **every** neuron in the next layer — this is called a **fully connected (dense) network**. No spatial structure, no memory of sequence — just pure weighted connections.

Core components inside each layer:

- Weights — control connection strength between neurons
- Bias — shifts the neuron's firing threshold
- Activation function (e.g. ReLU) — introduces non-linearity, without it the whole network collapses into one big linear equation regardless of depth

Advantages

- **Simple to understand and implement** — the most basic neural architecture, good starting point for learning core concepts (backprop, gradient descent, loss).
- **Universal approximator** — theoretically, a large enough ANN can approximate almost any function given enough neurons/layers.
- **Works well on structured/tabular data** — where there's no spatial or sequential relationship between features (e.g. rows in a spreadsheet).
- **Fast to train on small, simple datasets** compared to more complex architectures.

Disadvantages

- **No spatial awareness** — treats every pixel/feature as independent, ignoring relationships between neighboring values. This is exactly why your model's confidence dropped on the noisy "3" — it couldn't tell "meaningful edge" from "random noise pixel."
- **No memory** — can't handle sequences or time-dependent data (can't understand "this word depends on the previous word").
- **Parameter-heavy for large inputs** — a 784-pixel input to a 128-neuron layer alone is ~100,000 weights; scale to a real photo (e.g. $224 \times 224 \times 3$) and this explodes into millions of parameters, becoming inefficient and prone to overfitting.
- **Doesn't scale well to images/sequences** — loses spatial/temporal structure entirely by flattening input.

Comparison with Other Architectures

	ANN	CNN	RNN/LSTM	Transformer
Best for	Tabular/structured data	Images, spatial data	Sequences, time-series	Sequences, long-range context
Key mechanism	Fully connected layers	Convolution + pooling (local feature detection)	Hidden state (memory across time steps)	Self-attention (all positions see all positions)
Handles spatial structure?	✗ No	✓ Yes	✗ No	⚠ Only via positional encoding
Handles sequential/order data?	✗ No	✗ No	✓ Yes	✓ Yes (better, in parallel)
Parameter efficiency on images	✗ Poor (flattens, huge param count)	✓ Efficient (shared filters)	N/A	Efficient but computationally heavy
Training speed	Fast (simple data)	Moderate	Slow (sequential processing, can't parallelize easily)	Fast (parallelizable, unlike RNN)
Known weakness	No structure awareness	Limited long-range context	Vanishing gradients on long sequences	High compute/memory cost

One-line summary of the progression (matches sprint roadmap exactly):

ANN handles basic pattern learning → CNN adds spatial awareness for images → RNN/LSTM adds memory for sequences → Transformer replaces memory with attention, seeing the whole sequence at once.

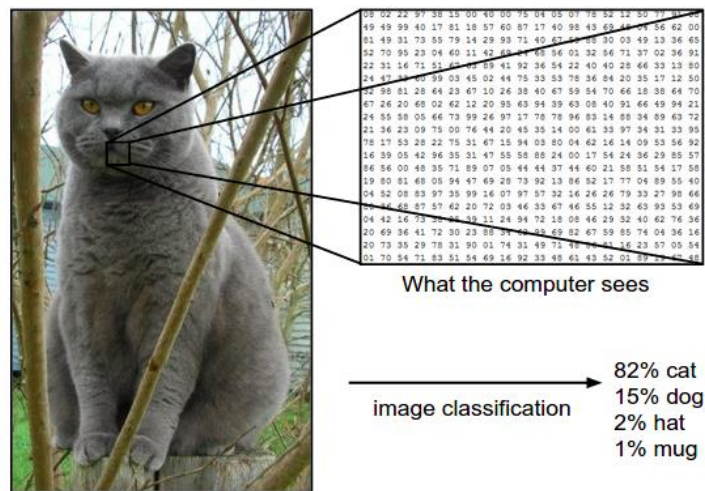
This positions you perfectly for Days 3-4: CNNs exist specifically to solve ANN's "no spatial awareness" weakness — which you already saw firsthand with our noisy digit test.

CNN - Convolutional Neural Networks

Similar to how a child learns to recognise objects, we need to show an algorithm millions of pictures before it is able to generalize the input and make predictions for images it has never seen before.

Computers 'see' in a different way than we do. Their world consists of only numbers. Every image can be represented as 2-dimensional arrays of numbers, known as pixels.

But the fact that they perceive images in a different way, doesn't mean we can't train them to recognize patterns, like we do. We just have to think of what an image is in a different way.



CNN — Biological Origin

To recognize objects in images, we use a Convolutional Neural Network (CNN) — named after its core operation: **convolution**.

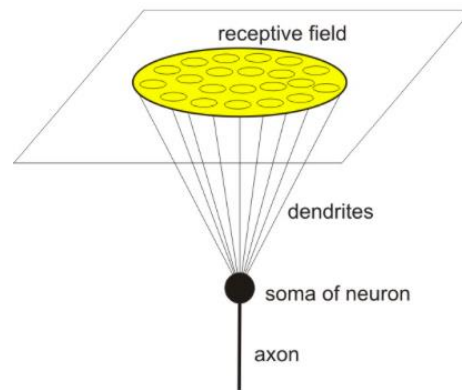
Inspired by the brain: Research by D.H. Hubel and T.N. Wiesel (1950s-60s) on cat/monkey visual cortexes found neurons that respond exclusively to stimuli in their direct environment.

Two neuron types they identified:

- **Simple cells (S cells)** — activate on basic shapes: lines at a fixed position and specific angle.
- **Complex cells (C cells)** — larger receptive fields; **not** sensitive to exact position. They keep responding to a stimulus even if its position on the retina shifts. ("Complex" here = more flexible, not more complicated.)

Receptive field (key term): the specific region of the retina that affects a given neuron's firing. Every sensory neuron has a similar receptive field, and these fields overlap with each other.

Why this matters for CNNs: this S-cell/C-cell distinction directly maps onto CNN design later — filters detecting simple local patterns (like S cells), and pooling providing position-flexibility (like C cells). That's the biological seed for why CNNs work the way they do.



CNN — Hierarchy & History

Hierarchy in the brain: Information is stored in sequential patterns. The **neocortex** (brain's outermost layer) stores information hierarchically, organized into **cortical columns** — uniform groupings of neurons.

1980 — Neocognitron (Fukushima): First hierarchical neural network model, directly inspired by S-cells/C-cells. Could recognize patterns by learning object *shapes*.

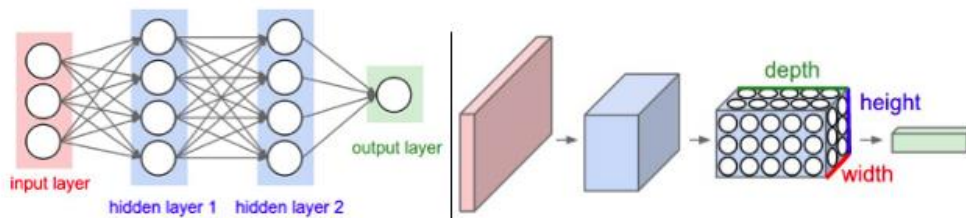
1998 — LeNet-5 (Bengio, Le Cun, Bottou, Haffner): The first true Convolutional Neural Network. Could classify handwritten digits — the direct ancestor of what you're about to build on MNIST yourself.

CNN — Architecture (vs Regular ANN)

Regular ANN: Input passes through hidden layers, where every neuron is **fully connected** to every neuron in the previous layer. Final fully-connected layer = output/predictions.

CNN — three key differences:

1. **3D layers** — organized by width, height, **and depth** (not just a flat 1D stack like ANN).
2. **Local connections only** — a neuron connects to just a **small region** of the previous layer, not all of it (unlike ANN's full connectivity).
3. **Output = probability vector along depth** — final result is reduced to a single vector of class probability scores.



Left: A regular 3-layer Neural Network. Right: A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

CNN — The Two Core Components

1. Feature Extraction (Hidden layers)

Performs convolutions + pooling to **detect features**. Example: for a zebra image, this part detects stripes, ears, legs — the raw visual building blocks, not the final label yet.

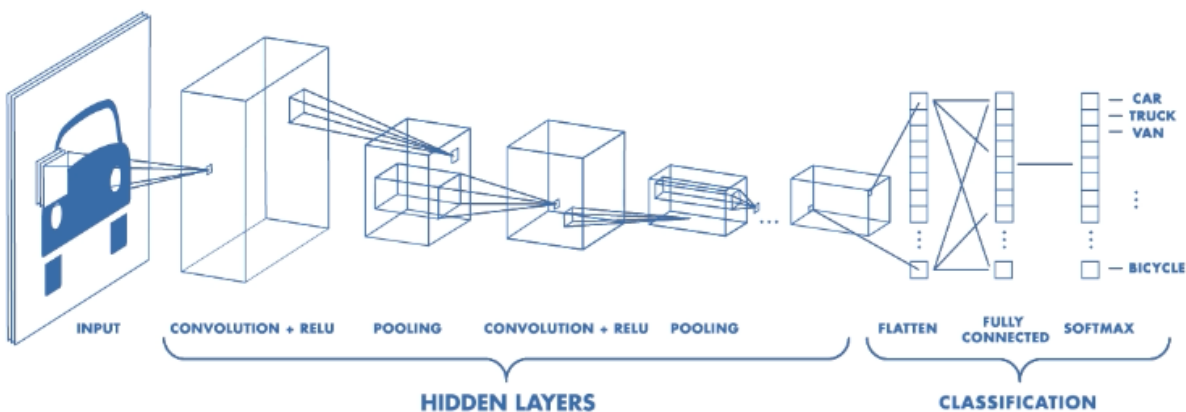
2. Classification

Fully connected layers act as a **classifier on top of** the extracted features — assigning a probability score for what the object actually is.

before we start building we import the libraries

```
import numpy as np
```

```
from keras.layers import Conv2D, Activation, MaxPool2D, Flatten, Dense
from keras.models import Sequential
```



CNN — Convolution Operation

Convolution (term): mathematical combination of two functions to produce a third — merges two sets of information.

In CNN terms: convolution = applying a **filter/kernel** (same thing, interchangeable terms) to the input data → produces a **feature map**.

How it works, step by step:

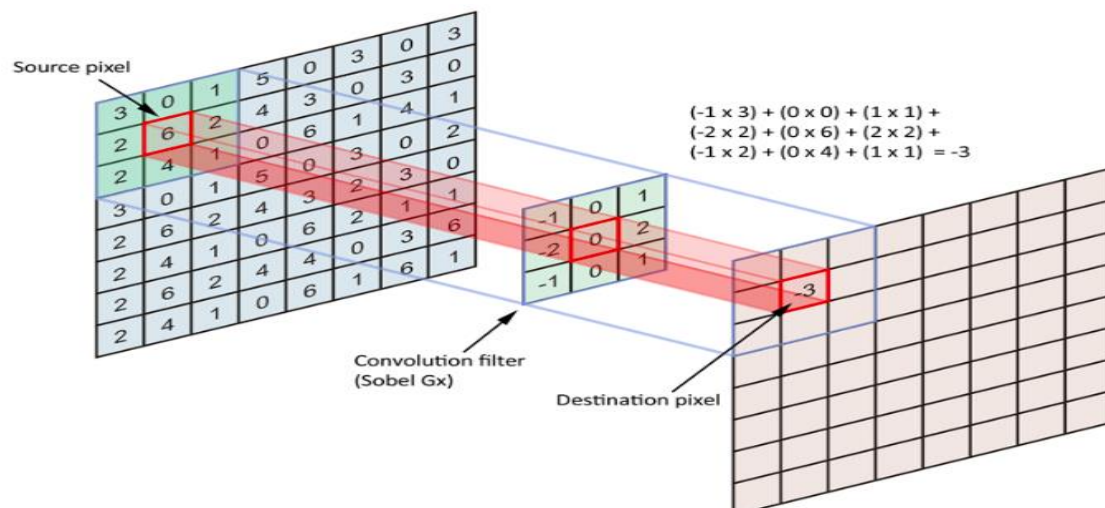
1. Filter slides over the input.
2. At every position, a **matrix multiplication** happens between filter and the covered input region.
3. The result is **summed** and placed onto the feature map.

Key term — Receptive field: the area covered by the filter at each step. Named directly after the biological receptive fields (S-cells/C-cells) from earlier. Example filter size: **3×3**.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

CNN — Convolution in 3D (Real Images) The 2D explanation was simplified for teaching. **Real convolutions happen in 3D - Why:** every image is a **3D matrix** - width, height, and **depth**. Depth = the **color channels** (RGB = 3 channels: Red, Green, Blue).



The filter slides over the input and performs its output on the new layer.

CNN — Multiple Filters, Activation, Stride

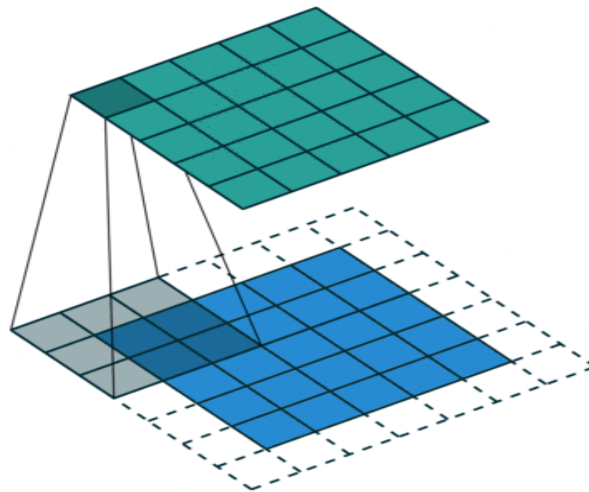
Multiple filters: many convolutions run on the same input, each using a **different filter** → produces **multiple feature maps**. All of these stack together as the final output of that convolution layer (this is the "depth" of the *next* layer).

Activation function (same role as in ANN): applied to convolution output to introduce **non-linearity**. Typically, **ReLU**.

Stride (key term): the step size the filter moves each slide.

- **Stride = 1** (default/common) → filter moves pixel by pixel, lots of overlap.
- **Larger stride** → bigger jumps, less overlap, smaller output feature map.

Connects back to your Sprint 03 hyperparameter list: stride is one of the 4 key CNN hyperparameters (kernel size, filter count, stride, padding) — this is the exact term.



CNN — Padding & Pooling

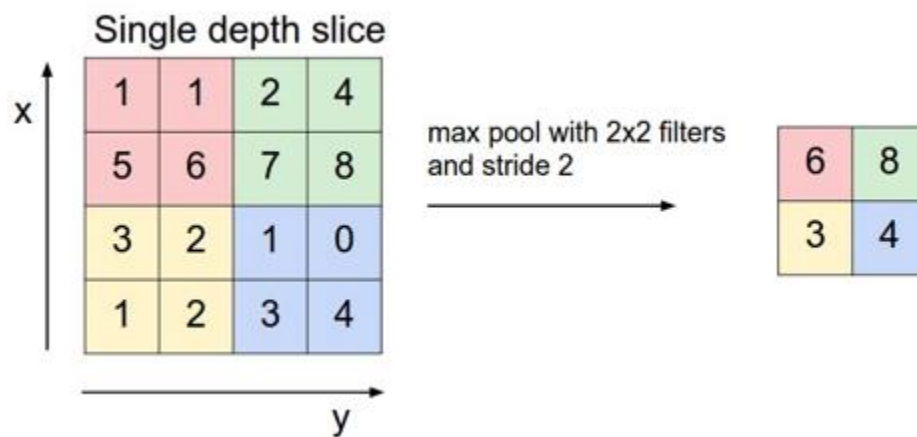
Problem: feature maps shrink after convolution (always smaller than the input).

Padding (key term): adds a layer of **zero-value pixels** around the input.

- Keeps spatial size constant after convolution.
- Also improves performance and ensures kernel + stride fit properly onto the input.

Pooling: added after a convolution layer. Purpose: **reduce dimensionality** → fewer parameters, less computation → faster training + controls **overfitting**.

Max pooling (most common type): takes the **maximum value** in each window (window size set beforehand). Shrinks feature map size while keeping the most significant information.



Max pooling takes the largest values.

CNN — 4 Key Hyperparameters (Summary)

When designing a CNN, these 4 must be decided:

1. **Kernel size** — dimensions of the filter (e.g. 3×3).
2. **Filter count** — how many different filters to use per layer.
3. **Stride** — step size the filter moves each slide.
4. **Padding** — whether/how much zero-padding is added around the input.

Full concept map so far (everything covered up to this point):

Concept	One-line meaning
Convolution	Filter slides over input, produces feature map
Receptive field	Area covered by the filter at each step
Filter/Kernel	The sliding matrix that detects a specific pattern
Depth (3D)	Color channels (RGB=3, grayscale=1)
Multiple filters	Each detects a different feature → multiple feature maps
Activation (ReLU)	Adds non-linearity after convolution
Stride	Step size of filter movement
Padding	Zero-pixels added to prevent shrinking
Pooling (max)	Shrinks feature map on purpose, keeps key info
Hyperparameters	Kernel size, filter count, stride, padding

```
# Images fed into this model are 512 x 512 pixels with 3 channels
```

```
img_shape = (28,28,1)
```

```
# Set up the model
```

```
model = Sequential()
```

```
# Add convolutional layer with 3, 3 by 3 filters and a stride size of 1# Set padding so that input size equals output size
```

```
model.add(Conv2D(6,2,input_shape=img_shape))
```

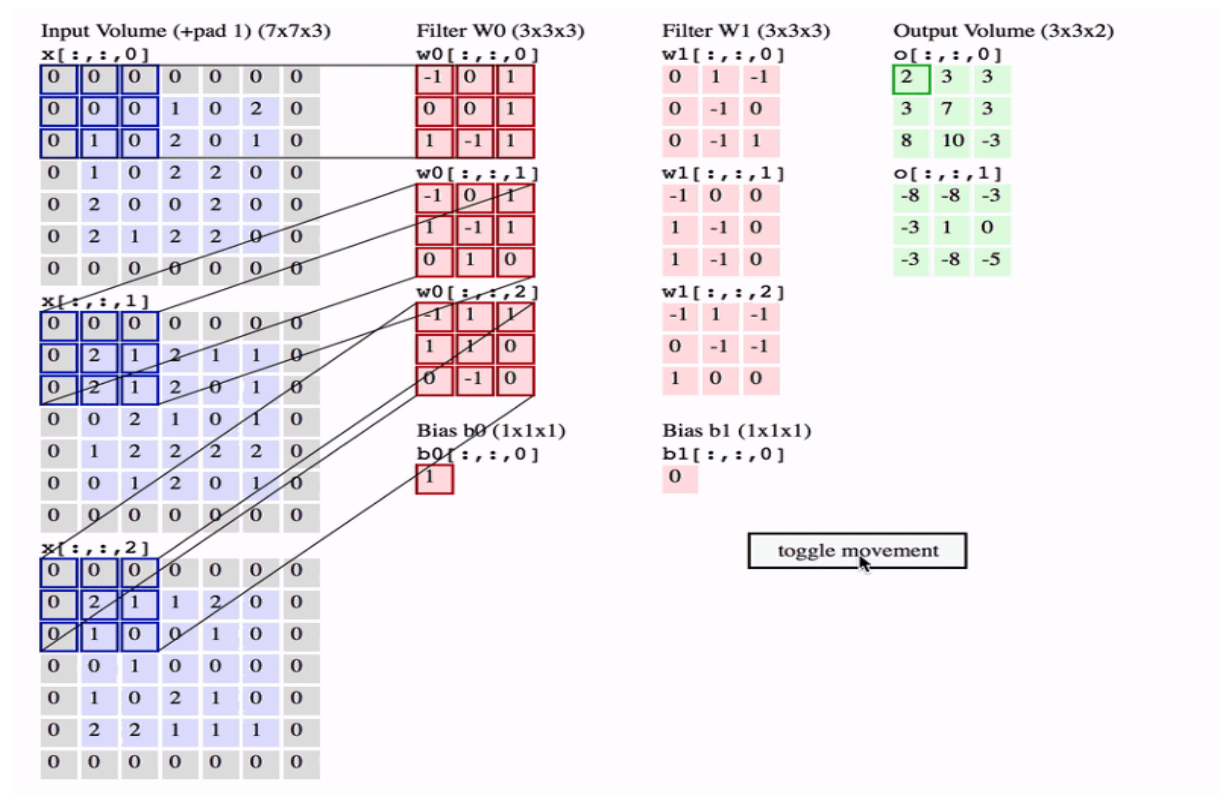
```
# Add relu activation to the layer
```

```
model.add(Activation('relu'))
```

```
#Pooling
```

```
model.add(MaxPool2D(2))
```

A nice way of visualizing a convolution layer is shown below. Try to look at it for a bit and really understand what is happening



How convolution works with $K = 2$ filters, each with a spatial extent $F = 3$, stride, $S = 2$, and input padding $P = 1$.

CNN — Classification Part

The problem: fully connected layers can only accept **1D data** — but after convolution/pooling, data is still **3D**.

Flatten (key term): converts 3D volume $\rightarrow$ 1D vector. Purely a reshaping step, no learning happens here.

Fully connected layers (final part): neurons here connect to **all** activations in the previous layer — same as a regular ANN.

Full picture, tied together:

Feature extraction (Conv + Pool, 3D, local connections) $\rightarrow$ **Flatten** (3D $\rightarrow$ 1D) $\rightarrow$ Fully connected layers (same as ANN, full connections) $\rightarrow$ Output probabilities

Key insight worth noting: a CNN isn't a totally different beast from ANN — it's ANN's classification logic *bolted onto* a specialized 3D feature-detection front end. That's exactly why the last part "is in principle the same as a regular Neural Network" — the flatten step is literally the handoff point between CNN-specific logic and ANN-specific logic.

#Fully connected layers

Use Flatten to convert 3D data to 1Dmodel.add(Flatten())

Add dense layer with 10 neuronsmodel.add(Dense(10))

we use the softmax activation function for our last layermodel.add(Activation('softmax'))

give an overview of our model

model.summary

		Layer (type)
Output Shape	Param #	
		conv2d_1 (Conv2D)
(None, 27, 27, 6)	30	activation_1(Activation)

[illegible]

CNN — Training

Training works the **same way** as a regular neural network — via **backpropagation** and **gradient descent**.

Key difference: mathematically more complex, because gradients now have to flow back through the **convolution operations** too (not just simple weighted sums like ANN).

Full article, fully distilled — complete map:

Section	Core idea
Biological origin	S-cells/C-cells inspire local, position-flexible detection
Hierarchy & history	Neocognitron (1980) → LeNet-5 (1998, first real CNN)
Architecture vs ANN	3D layers, local connections, not fully connected
Two components	Feature extraction (detect patterns) → Classification (predict)
Convolution	Filter slides, multiplies, sums → feature map
3D convolution	Depth = color channels (RGB=3, grayscale=1)
Multiple filters + activation	Different filters → different feature maps; ReLU adds non-linearity
Stride	Step size of filter movement
Padding vs Pooling	Padding prevents shrinking; pooling shrinks on purpose
4 hyperparameters	Kernel size, filter count, stride, padding
Flatten	3D → 1D handoff point
Fully connected	Same as ANN — full connections, final prediction
Training	Same backprop/gradient descent, more complex due to convolution

That's the entire article, section by section, in your notes format. You now have full **understanding-level** coverage of CNNs — no code touched yet, exactly as you wanted.

""Before the training process, we have to put together a learning process in a particular form. It consists of 3 elements: an optimiser, a loss function and a metric.""

```
model.compile(loss='sparse_categorical_crossentropy', optimizer = 'adam', metrics=['acc'])
```

```
# dataset with handwritten digits to train the model onfrom keras.datasets import mnist
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = np.expand_dims(x_train,-1)
```

```
x_test = np.expand_dims(x_test,-1)
```

```
# Train the model, iterating on the data in batches of 32 samples# for 10 epochs
```

```
model.fit(x_train, y_train, batch_size=32, epochs=10, validation_data=(x_test,y_test))
```

```
# Training...
```

Train on 60000 samples, validate on 10000 samplesEpoch 1/1060000/60000

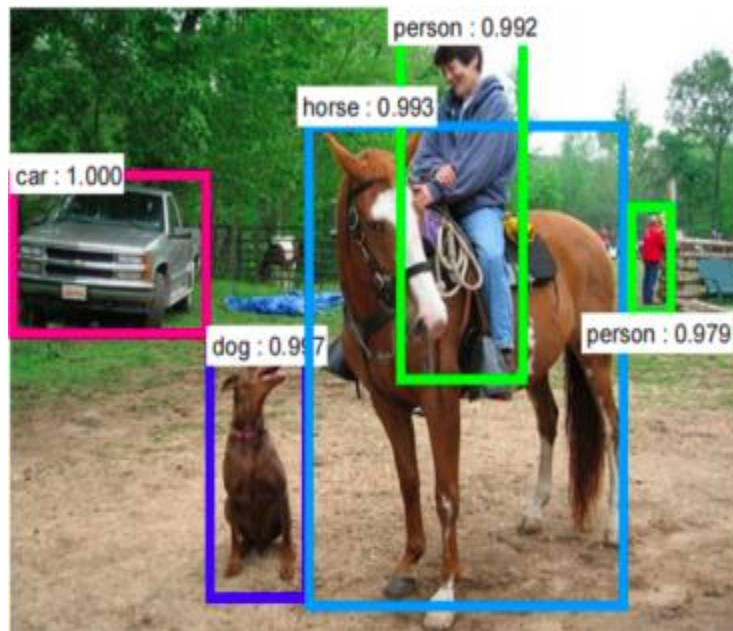
```
[=====] - 10s 175us/step - loss: 4.0330 - acc: 0.7424 - val_loss: 3.5352 - val_acc: 0.7746Epoch 2/1060000/60000 [=====] - 10s 169us/step - loss: 3.5208 - acc: 0.7746 - val_loss: 3.4403 - val_acc: 0.7794Epoch 3/1060000/60000 [=====] - 11s 176us/step - loss: 2.4443 - acc: 0.8372 - val_loss: 1.9846 - val_acc: 0.8645Epoch 4/1060000/60000 [=====] - 10s 173us/step - loss: 1.8943 - acc: 0.8691 - val_loss: 1.8478 - val_acc: 0.8713Epoch 5/1060000/60000 [=====] - 10s 174us/step - loss: 1.7726 - acc: 0.8735 - val_loss: 1.7595 - val_acc: 0.8718Epoch 6/1060000/60000 [=====] - 10s 174us/step - loss: 1.6943 - acc: 0.8765 - val_loss: 1.7150 - val_acc: 0.8745Epoch 7/1060000/60000 [=====] - 10s 173us/step - loss: 1.6765 - acc: 0.8777 - val_loss: 1.7268 - val_acc: 0.8688Epoch 8/1060000/60000 [=====] - 10s 173us/step - loss: 1.6676 - acc: 0.8799 - val_loss: 1.7110 - val_acc: 0.8749Epoch 9/1060000/60000 [=====] - 10s 172us/step - loss: 1.4759 - acc: 0.8888 - val_loss: 0.1346 - val_acc: 0.9597Epoch 10/1060000/60000 [=====] - 11s 177us/step - loss: 0.1026 - acc: 0.9681 - val_loss: 0.1144 - val_acc: 0.9693
```

CNN — Article Summary

CNNs are especially useful for **image classification and recognition**, built on two main parts: **feature extraction + classification**.

Core technique: convolution — filter slides over input, merges input value + filter value onto the feature map.

End goal: feed new images to the CNN so it can output a **probability** for what it thinks the object is (or even describe the image with text — a hint toward captioning models, further ahead than this sprint).



CNN — Article's Keras Code (Reference Only)

This is the article's own implementation — in **Keras**, not PyTorch (the framework we chose). Kept here for reference/understanding, not what we'll build.

Code line	Maps to concept
<code>img_shape = (28,28,1)</code>	Input shape: 28×28, depth=1 (grayscale)
<code>Conv2D(6,2, input_shape=img_shape)</code>	6 filters, kernel size 2×2
<code>Activation('relu')</code>	ReLU non-linearity after convolution
<code>MaxPool2D(2)</code>	Max pooling, 2×2 window
<code>Flatten()</code>	3D → 1D conversion
<code>Dense(10)</code>	Fully connected output layer, 10 neurons (digits 0-9)
<code>Activation('softmax')</code>	Converts final scores → probabilities
<code>model.compile(loss=..., optimizer='adam', metrics=['acc'])</code>	Loss function + optimizer + metric, set before training
<code>model.fit(x_train, y_train, batch_size=32, epochs=10, ...)</code>	Training — hidden loop (Keras handles forward/backward/update internally)

Note on `input_shape=(28,28,1)`: the code comment above it says "512×512 pixels" — that's a leftover/mismatched comment from the article, the actual shape used is 28×28×1 for MNIST. Worth knowing when reading it, not an error on your part.

This confirms the exact same 8-step algorithm from before — just compressed into 10 lines of Keras, with steps 1-3 hidden inside `Conv2D/MaxPool2D/.fit()` instead of written out manually.

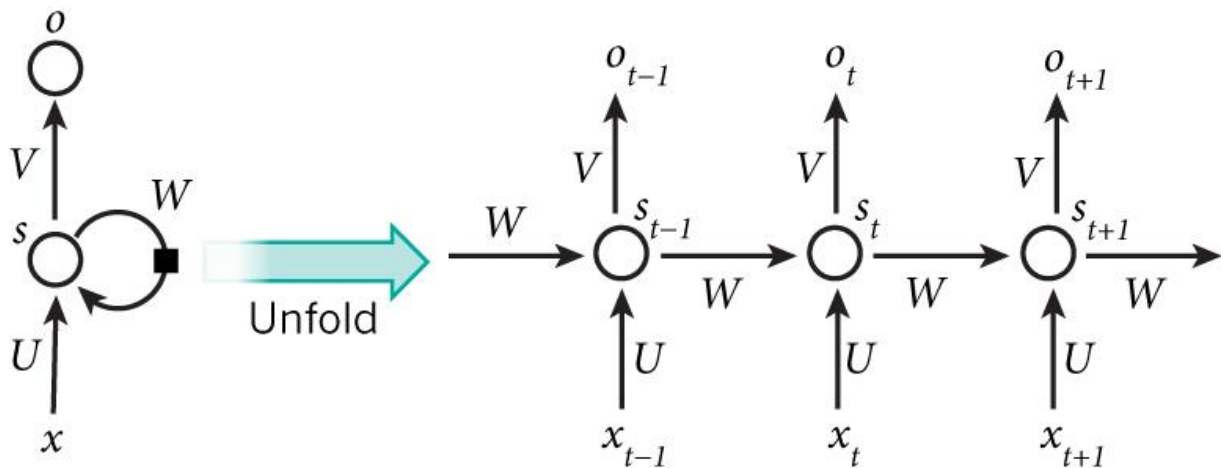
RNN-Recurrent Neural Networks

A **neural network** that feeds its previous output back in as input to the next step — giving it a form of memory.

- Regular neural nets treat each input independently, which fails for language (you need context from earlier words to predict the next one).
- RNNs solve this with a **hidden state** that carries information from previous steps forward.
- This hidden state acts like short-term memory of what's been seen so far.
- In theory this memory can span any sequence length — in practice, it fades after just a few steps (this is the vanishing gradient issue you'll hit next).

That last point is your natural bridge into LSTM/GRU.

Diagrammatic Representation



This diagram is the classic "unfolded RNN" picture.

- **Left side** is the compact loop view: one cell that feeds back into itself over time.
- **Right side** is that same cell "unfolded" across time steps — one copy per input, so a 4-word sentence becomes a 4-step chain.
- **The key insight:** it's the *same* U , V , W weights reused at every step (not new weights per layer like a normal deep net). That's what "recurrent" means, and why RNNs have far fewer parameters than a stacked feedforward network would need for the same job.
- **Three roles:**
 - U : input $\rightarrow$ hidden state ($x_t \rightarrow s_t$)
 - W : previous hidden state $\rightarrow$ current hidden state ($s_{t-1} \rightarrow s_t$) — this is the memory link
 - V : hidden state $\rightarrow$ output ($s_t \rightarrow o_t$)

So each s_t is a blend of "what I just saw" (via U) and "what I remembered" (via W) — that blended state is the memory.

Next natural step: the actual update equation ($s_t = \tanh(U \cdot x_t + W \cdot s_{t-1})$), then why repeatedly multiplying by W over many steps causes vanishing gradients. Want that next?

That's exactly right - three formulas, three jobs:

- $h_t = f(h_{t-1}, x_t)$ — general form: current state depends on previous state + current input
- $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$ — the actual computation: blend previous memory (W_{hh}) with new input (W_{xh}), squash with $\tanh$
- $O_t = W_{hy} \cdot h_t$ — output is just a linear projection of the current hidden state

Note this maps directly onto the diagram from before: $W_{xh} = U$, $W_{hh} = W$, $W_{hy} = V$ — just different notation for the same three weight matrices.

One thing worth locking in: **$\tanh$ keeps values between -1 and 1**, which is partly why repeated multiplication across many time steps shrinks gradients toward zero — that's the vanishing gradient problem you'll hit right after this.

Backpropagation Through Time (BPTT)

- $L = -\ln(p_c)$ — you only care about the probability the model assigned to the *correct* class.
- If it's confident and right (0.95), loss is small (0.051). If it's confident and wrong, loss blows up (e.g., predicting 0.05 for the correct class gives $L = -\ln(0.05) \approx 3.0$).
- This loss then gets backpropagated — but in an RNN this is called **BPTT (Backpropagation Through Time)**: since the network is "unfolded" across time steps, gradients flow backward through every step, and the *same* shared weights (U , W , V) get gradient contributions from every single time step, which are then summed up.
- This is exactly where the vanishing gradient problem bites: gradients get multiplied by W repeatedly as they flow back through each step, so for long sequences those early-step gradients shrink toward zero and the network "forgets" long-range dependencies.

Steps for Backpropagation (BPTT)

1. **Compute loss** at each time step using cross-entropy (comparing predicted vs. actual output), with the network unfolded across all time steps.
2. **Compute gradients** for each time step, with respect to each weight parameter (U , W , V).
3. **Sum the gradients** across all time steps — since the weights are shared, every step's contribution gets added together into one total gradient per weight.
4. **Update the weights** — both the recurrent weight (W_{hh}) and the dense/output layer weights (W_{xh} , W_{hy}) — using gradient descent.

That gradient-summing step (3) is exactly why long sequences cause trouble: many small (or vanishing) per-step gradients get added, but the earliest steps' contributions are so tiny they barely move the weights — the network effectively stops learning from distant past inputs.

Vanishing & Exploding Gradients

- Gradients flow back through the unfolded network via the chain rule, which means **repeated matrix multiplication** across time steps.
- If those values are < 1 , multiplying them together shrinks the gradient toward zero the further back you go → **vanishing gradient** → early time steps stop learning (this is why RNNs "forget" long-range context).
- If those values are > 1 , multiplying repeatedly makes the gradient blow up → **exploding gradient** → unstable training, weights become huge or NaN.

Quick fixes worth knowing:

- Exploding → **gradient clipping** (cap the gradient at a max value).
- Vanishing → this is the deeper problem, and it's what **LSTM/GRU** were specifically designed to fix, via gating mechanisms that let gradients flow more directly.

RNN Architecture Types

- **One-to-one**: Fixed-size input → fixed-size output, no sequence involved — essentially a single-layer network.
Example: Classifying one photo as "cat" or "dog."
- **One-to-many**: A single input feeds in once, then the model generates a sequence of outputs, each depending on the previous one.
Example: Give a music AI one starting note, and it generates a full melody, note by note.
- **Many-to-one**: A sequence of inputs is processed step by step, but only one final output is produced at the end.
Example: Feed in an entire product review word by word — the model outputs one verdict: **Negative**.
- **Many-to-many (same length)**: One output is produced for every input, in sync, frame-by-frame or step-by-step.
Example: A security camera feed labels each frame in real time — "person," "empty," "car."
- **Many-to-many (different length)**: The full input sequence is read first, then a separate output sequence is generated — lengths don't have to match.
Example: Translating "How are you?" → "Comment ça va?"

Bidirectional RNNs

Concept: A regular RNN only sees past context (left-to-right). A bidirectional RNN runs **two hidden layers in opposite directions** — one forward, one backward — and combines both at each time step, so the output depends on both past *and* future context.

Example: In the sentence "I like programming," the forward layer reads it as "I → like → programming," while the backward layer reads it reversed: "programming → like → I." At each word, the two directions' hidden states are concatenated to produce that word's output — meaning the model's understanding of "like" is informed by both what came before ("I") and what comes after ("programming").

This matters for tasks like **fill-in-the-blank** ("I ____ programming") where knowing the word *after* the blank helps predict the word *in* it — something a one-directional RNN can't use.

Quick RNN recap (your notes, confirmed correct):

- Remembers info via hidden state across time steps.
- Shares parameters across steps → fewer parameters to train.
- Handles variable-length input/output.
- Struggles with very long sequences (tanh/ReLU) and suffers vanishing/exploding gradients.

Applications of RNN

- **Text summarization:** reads a long article (many inputs) → produces a short summary (many-to-many, variable length) — e.g., a news site auto-generating headlines/briefs for each article.
- **Text recommendation:** predicts the next word/sentence as you type (one-to-many style generation) — e.g., autofill or autocomplete in email/data-entry tools.
- **Image recognition + description:** CNN extracts visual features from the image, then an RNN generates a caption word-by-word — e.g., "a dog running on grass" — combining one-to-one (image) with one-to-many (caption generation).
- **Music generation:** feed in a starting tune, RNN generates new notes step-by-step, each depending on the notes before it (one-to-many).

LSTM – Long Short-Term Memory

LSTM — The Problem of Long-Term Dependencies (Part 1)

RNN's appeal: connect past information to the present task — e.g. using earlier video frames to understand a current one.

Where RNN actually works fine: when the relevant context is **close by**. Example: predicting the last word in *"the clouds are in the sky"* — the answer ("sky") is obvious from just the immediately preceding words. **Small gap between relevant info and where it's needed → RNN handles it fine.**

LSTM — The Problem of Long-Term Dependencies (Part 2)

The harder case: *"I grew up in France... I speak fluent French."* Predicting the last word needs the word "language" nearby — but predicting *which* language needs "France," way back at the start. **Large gap between relevant info and where it's needed.**

The core RNN failure: as that gap grows, RNNs become unable to learn to connect the information.

LSTM — The Problem of Long-Term Dependencies (Part 3)

In theory: RNNs *can* handle long-term dependencies — a human could hand-pick the right parameters to solve toy examples of this.

In practice: they don't learn this on their own. Investigated in depth by Hochreiter (1991) and Bengio et al. (1994), who found fundamental reasons why it's difficult — this is the formal origin of what we've been calling the vanishing gradient problem.

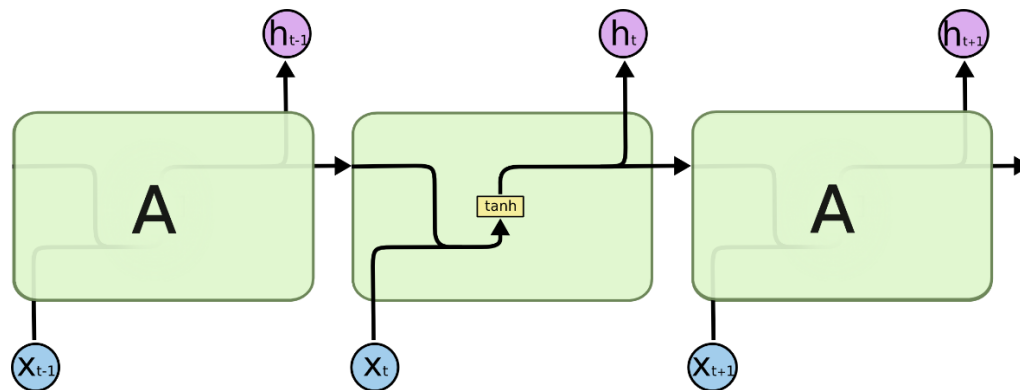
The fix: LSTMs don't have this problem.

LSTM Networks — Introduction

What LSTMs are: a special kind of RNN, capable of learning long-term dependencies. Introduced by Hochreiter & Schmidhuber (1997), refined and popularized since.

Core design philosophy: LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods is their **default behavior**, not something they struggle to learn.

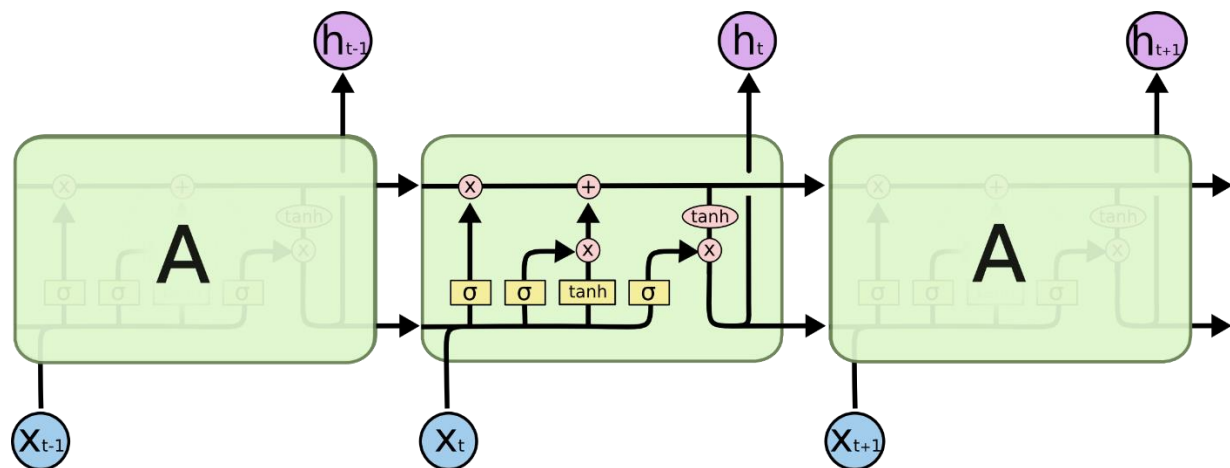
Structural starting point: all RNNs are a chain of repeating modules. In a **standard RNN**, each repeating module is very simple — just a single tanh layer.



LSTM Networks — Repeating Module Structure

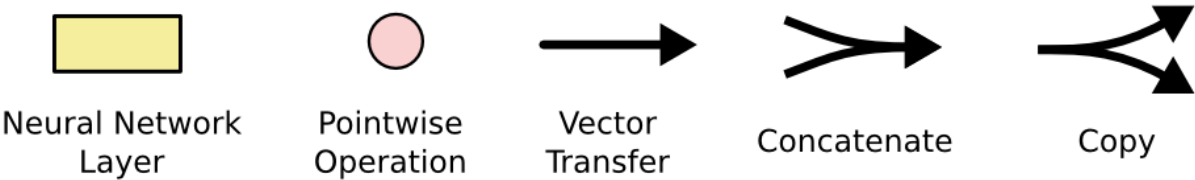
Standard RNN's repeating module: a single layer (just one tanh, as covered).

LSTM's repeating module: same chain-like structure overall, but instead of one layer, it has **four**, interacting in a very special way.



LSTM Networks — Notation Setup

Where we are: the article is about to walk through the LSTM diagram in detail, step by step. Before that, it pauses to get you comfortable with the **notation** used in the diagrams — not the mechanics yet, just how to read the visuals.

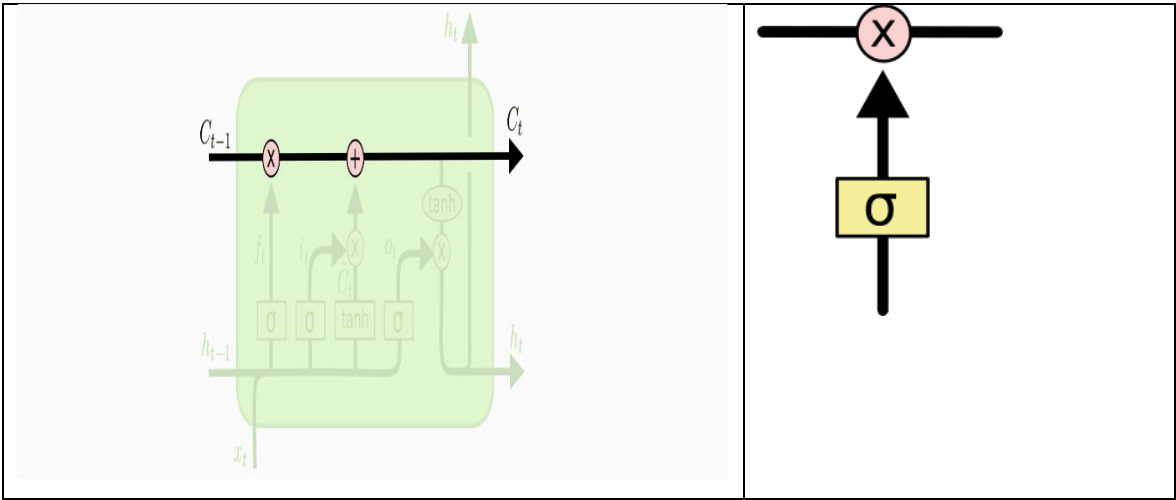


LSTM Networks — Diagram Notation (Legend)

Symbol	Meaning
Yellow box	A learned neural network layer (has trainable weights)
Pink circle	A pointwise operation (e.g. vector addition, multiplication — no learning, just math)
Line	An entire vector flowing from one node's output to another's input
Merging lines	Concatenation — two vectors joined into one
Forking line	Copy — the same content duplicated and sent to multiple destinations

LSTM Networks — The Core Idea (Cell State & Gates)

The key element: the **cell state** — the horizontal line running along the top of the diagram (labeled C_{t-1} to C_t).



Real-life example: it's a **conveyor belt** — it runs straight through the entire chain with only minor linear interactions. Information can flow along it essentially unchanged unless something deliberately alters it.

How the LSTM modifies it: by adding or removing information from the cell state — carefully regulated by structures called **gates**.

What a gate actually is (mechanically): a sigmoid neural network layer + a pointwise multiplication operation.

- Sigmoid outputs a number between 0 and 1 for each component.
- **0** = "let nothing through." **1** = "let everything through."

An LSTM has three of these gates, protecting and controlling the cell state.

LSTM — Gate 1: The Forget Gate

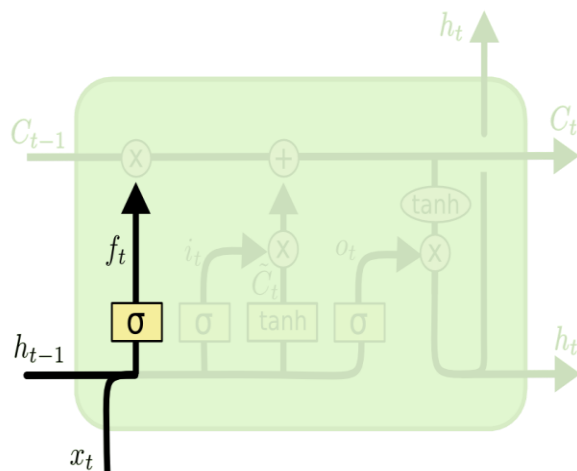
Its job: decide what to **throw away** from the cell state.

How it decides: the forget gate layer (a sigmoid) looks at h_{t-1} (previous hidden state) and x_t (current input), and outputs a number between 0 and 1 **for each number in the cell state**.

- **1** = completely keep this
- **0** = completely get rid of this

Formula: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

Real-life example (from the article): a language model tracking subject gender for pronoun agreement. When a **new subject** appears, the forget gate is what lets the model drop the **old** subject's gender — clearing space for the new, relevant fact instead.



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM — Gate 2: The Input Gate (Two-Part Process)

Its job: decide what **new** information to store in the cell state. Two parts:

Part A — Input gate layer (sigmoid): decides **which values** to update.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

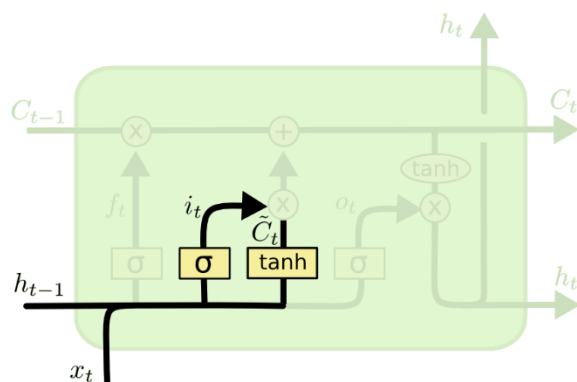
****Part B — Candidate layer (tanh):**** creates a vector of ****new candidate values**** that ***could*** be added to the cell state.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Why two separate parts, not one: sigmoid (i_t) answers "**how much** should we update?", while $\tanh(\tilde{C}_t)$ answers "**what** should we update it with?" These get combined in the next step to actually modify the cell state.

Real-life example (continuing the article's thread): when a new subject appears, this is the mechanism that adds the **new subject's gender** into the cell state — right after the forget gate cleared out the old one.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM — Updating the Cell State ($C_{t-1} \rightarrow C_t$)

The actual update step: the previous two gates already *decided* what to do — this step **executes** it.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

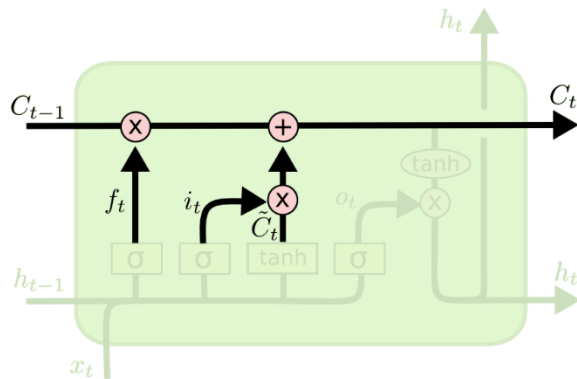
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Broken into two pieces:

- $f_t * C_{t-1}$ → **forget**: multiply old state by f_t , erasing whatever was decided to be forgotten

- $i_t * \tilde{C}_t \rightarrow$ **add**: bring in the new candidate values, scaled by how much they should matter

Real-life example (continuing the thread): this is the exact moment the old subject's gender actually gets **dropped**, and the new subject's gender actually gets **written in** — the forget gate and input gate did the *planning*, this line does the *doing*.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM — Gate 3: The Output Gate

Its job: decide what gets **output** as the hidden state h_t — a *filtered* version of the cell state, not the raw cell state itself.

Two-part process:

Part A — Output gate layer (sigmoid): decides **which parts** of the cell state to output.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

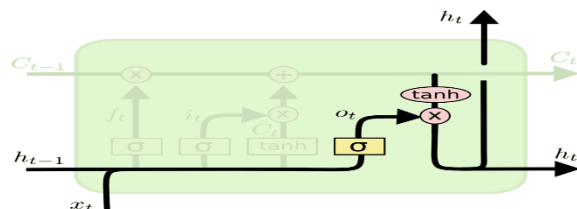
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Part B — Final filtering: push the cell state through tanh (squashing values to between -1 and 1), then multiply by o_t — so only the selected parts actually come through.

$$h_t = o_t * \tanh(C_t)$$

$$h_t = o_t * \tanh(C_t)$$

Real-life example (continuing the language model thread): having just seen a subject, the model might output whether it's **singular or plural** — not everything it knows, just the piece relevant for correctly conjugating whatever verb comes next.



$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

All three gates, now complete, in one line each:

- **Forget gate** — what do I erase from memory?
- **Input gate** — what new thing do I write into memory?
- **Output gate** — what part of memory do I actually reveal right now?

LSTM — Variant 1: Peephole Connections

Context: the LSTM described so far is the "standard" version - but almost every paper uses a slightly different variant. First one worth knowing:

Peephole connections (Gers & Schmidhuber, 2000): the gate layers are given direct access to look at the **cell state** too, not just h_{t-1} and x_t .

- **What changes in the math** (bold = the new addition):

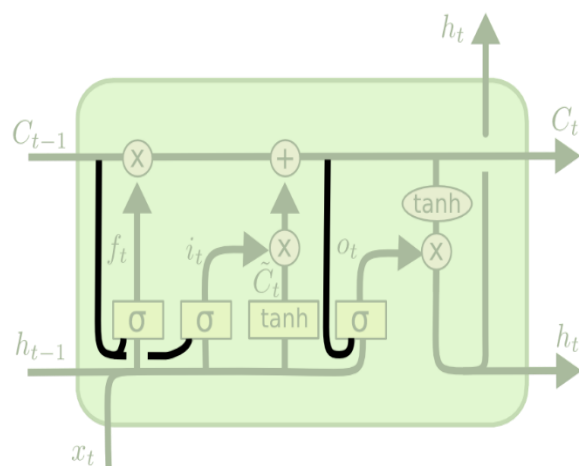
$$f_t = \sigma(W_f \cdot [\mathbf{C}_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [\mathbf{C}_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [\mathbf{C}_t, h_{t-1}, x_t] + b_o)$$

Intuition for why this helps: in the standard LSTM, gates decide what to forget/add/output based only on the *hidden state* and *current input* - they never directly "see" what's actually sitting in long-term memory. Peepholes let each gate **glance at the cell state itself** before deciding, similar to double-checking your notes before deciding what to erase, rather than deciding purely from memory of what you *think* you wrote.

- **One subtle but interesting detail:** the output gate's peephole uses C_t (the *already-updated* new state), while forget and input gates use C_{t-1} (the *old* state) - because forget/input happen *before* the update, and output happens *after*.



$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

LSTM — Variant 2: Coupled Forget & Input Gates

Standard LSTM: forget and input decisions are made **separately**.

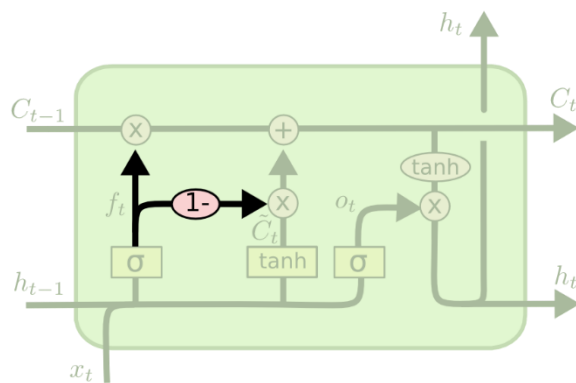
Coupled variant: the two decisions are made **together** — you only forget when you're about to input something in its place, and only input something new when you're forgetting something older.

New formula (replaces the separate f_t and i_t terms):

$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

Intuition: instead of independently asking "what do I erase?" and "what do I add?", it's one combined question: "how much of the old do I keep vs. how much of the new do I let in?" — like a single dial instead of two separate switches.



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

LSTM — Variant 3: GRU (Gated Recurrent Unit)

Introduced by Cho et al. (2014). Since you asked to hold off on GRU for now, here's just the **what**, briefly, for context - not the full mechanics:

- Combines the forget and input gates into a single **"update gate"** (z_t)
- **Merges** the cell state and hidden state into one — no separate C_t conveyor belt anymore, just h_t
- Result: **simpler** than LSTM, fewer parameters, and has grown increasingly popular

****GRU's formulas, for reference (not to implement yet, per your plan):****

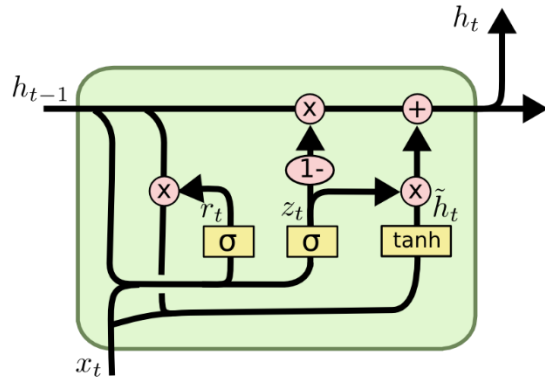
$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \text{ (update gate)}$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \text{ (update gate)}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \text{ (reset gate)} \quad \tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTM — Conclusion

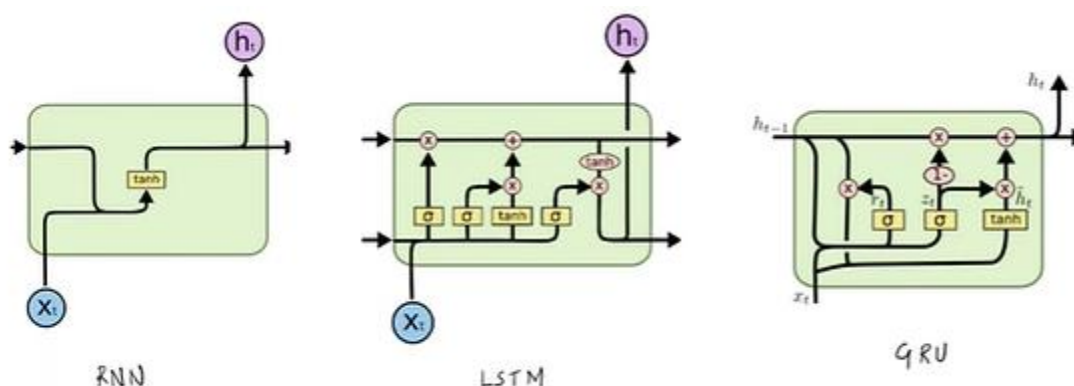
Closing summary: the remarkable RNN results mentioned earlier are essentially all achieved using LSTMs — they work a lot better for most tasks. As equations, LSTMs look intimidating, but walking through step-by-step (as we just did) makes them approachable.

The forward-looking hook — Attention: LSTMs were a big step forward for RNNs. The next big step, per common researcher opinion, is **attention** — letting every step of an RNN pick which information to look at from a larger pool, rather than relying on a single compressed hidden/cell state. Example given: an RNN captioning an image might choose a *different part of the image* to focus on for each word it generates (Xu et al., 2015).

Other research directions mentioned (context only, not needed for your sprint): Grid LSTMs, RNN-based generative models — the article closes by noting this has been an exciting, fast-moving area of research.

GRU - Gated Recurrent Units

What is a GRU? Gated Recurrent Units Explained (Architecture & Math)



GRU - Why It Exists (Evolution: RNN $\rightarrow$ LSTM $\rightarrow$ GRU)

The RNN problem: works fine on short sequences, but on longer ones suffers from vanishing/exploding gradients - it "forgets" information from the beginning by the time it reaches the end.

LSTM's fix, and its cost: solves this with a complex structure - **3 gates** - but that complexity means more parameters, making training computationally expensive on large datasets.

GRU's motivation (2014): built specifically for **efficiency**, not new capability. Uses only **2 gates** instead of LSTM's 3 $\rightarrow$ fewer parameters $\rightarrow$ faster training, especially valuable for large datasets or limited compute.

Performance - no clear winner:

- GRU can outperform LSTM on some tasks, LSTM wins on others
- Not a strict upgrade — a trade-off between performance and training speed
- In practice: **treat LSTM vs. GRU as a hyperparameter to test**, not a fixed choice

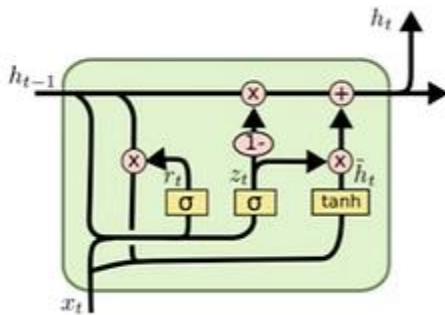
This directly validates your own results — GRU beat LSTM on your IMDB task (87.22% vs 83.65%), which is a completely normal, expected outcome per this article, not an anomaly.

GRU — Core Idea

LSTM's approach: two separate states — **cell state** (long-term) + **hidden state** (short-term) — managed by 3 gates.

GRU's simplification: merges both into **one single hidden state**, carrying short *and* long-term context together. Only **2 gates** needed to manage it: **Reset Gate** and **Update Gate**.

The Setup



GRU - The Goal & Terminology

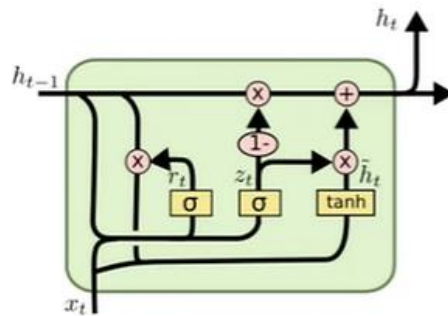
The goal, simply stated: at each timestamp t , take two inputs — previous hidden state h_{t-1} and current input x_t - and calculate the new hidden state h_t . That's the entire job.

Key terms (all vectors - lists of numbers):

Symbol	Meaning
h_{t-1}	Previous hidden state
h_t	Current hidden state (the output we're calculating)
x_t	Current input
r_t	Reset gate
z_t	Update gate
$\tilde{h}_t$	Candidate hidden state

Dimensionality rule (important, easy to miss): h , r , z , and $\tilde{h}$ must **all share the same dimensionality** — e.g. if h_{t-1} is 4-dimensional, then h_t , r_t , z_t , and $\tilde{h}_t$ must also be 4-dimensional. **Only x_t is exempt** — it can be a different size entirely.

GRU - Architecture Notation, Input Vectors & the 4-Step Process



Yellow boxes = Fully Connected layers. Node count is a hyperparameter (we choose it). σ -labeled boxes use Sigmoid; tanh-labeled use Tanh. **Key link:** the number of nodes in these layers = the exact dimensionality of h_t , r_t , etc.

Pointwise operations (element-wise, not matrix math):

Given $A = [a, b, c]$, $B = [d, e, f]$:

- Multiply: $[a \cdot d, b \cdot e, c \cdot f]$
- Add: $[a + d, b + e, c + f]$

The input x_t — worked example:

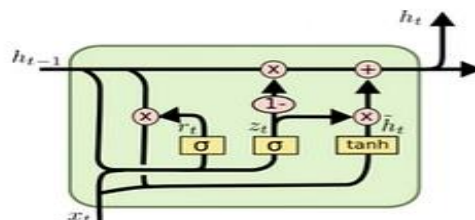
GRUs can't read raw text — words must become numbers first (**Text Vectorization:** One-Hot Encoding, Bag of Words, Word2Vec). Using One-Hot Encoding on a 3-word vocabulary (Cat, Mat, Rat):

- Cat $\rightarrow [1, 0, 0]$, Mat $\rightarrow [0, 1, 0]$, Rat $\rightarrow [0, 0, 1]$

A 3-word sentence = **3 timestamps**, one word processed per step — x_1 = first word's vector, x_2 = second word's, and so on.

GRU — What the Hidden State Actually Means (King's Story Analogy)

Real-life example, extended: the hidden state h_t = the system's evolving memory of a story. Imagine 4 dimensions with assigned meaning — [Power, Conflict, Tragedy, Revenge] — even though in a real network these dimensions are actually a **black box** (this is just for intuition).



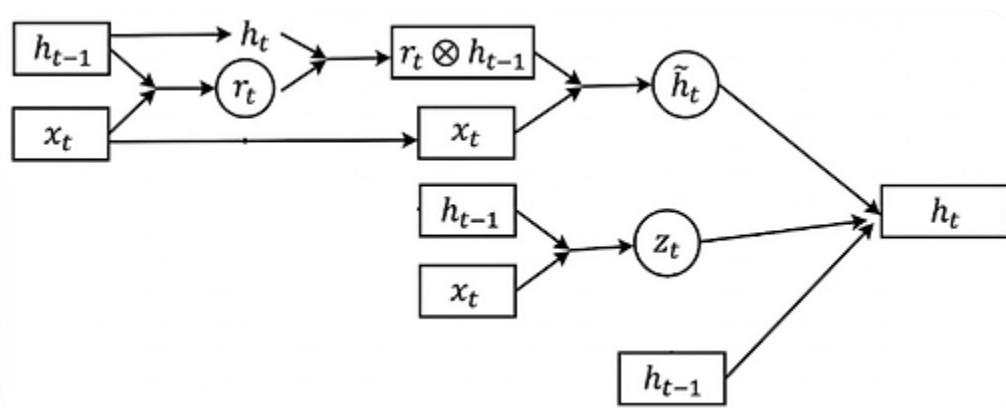
Tracing the story, word by word:

t	Sentence	Vector	What changed
1	"There was a strong king"	[0.9, 0, 0, 0]	Power established
2	"An enemy king appeared"	[1.0, 0.5, 0, 0]	Conflict rising
3	"War — king killed"	[0.7, 0.7, 0.4, 0]	Power drops, Tragedy enters
4	"Son grows strong"	[0.9, 0.6, 0.3, 0]	Power rebuilds, Tragedy fades slightly
5	"Son also killed, history repeats"	[0.8, 0.8, 0.6, 0.5]	Tragedy spikes, Revenge seed appears
6	"Grandson kills enemy king, revenge complete"	[0.7, 0.8, 0.3, 1.0]	Revenge fully realized

The key insight: the GRU doesn't just remember the *last* word — it carries a **weighted, evolving summary** of the entire narrative, updated (not overwritten) at every step.

Direct link back to your own IMDB results: this is exactly why your GRU handled the "long negative-buildup, positive twist" review differently than a plain RNN would — the hidden state was carrying an evolving *summary* (like this Power/Conflict/Tragedy/Revenge example) rather than a fragile, easily-erased single value. It's also exactly why your GRU's 57.78% confidence on that review makes sense — the "summary" ended up genuinely mixed, not confidently one way or the other, much like how t=6 in this story still holds some Conflict and Tragedy alongside the resolved Revenge.

GRU — The Two-Phase Mechanism (Reset Gate vs Update Gate)



Phase 1 — Creating the Candidate Hidden State ($\tilde{h}_t$)

Goal: draft a "proposed" new memory, based heavily on the current input.

Tool: the **Reset Gate** (r_t).

Logic: looks at h_{t-1} and x_t , decides how much of the *past* is relevant to *this* input. On a big topic shift, the Reset Gate can tell the model to largely ignore old memory while drafting this candidate.

Phase 2 — Finalizing the Current Hidden State (h_t)

Goal: decide the *actual* final memory for this timestamp.

Tool: the **Update Gate** (z_t).

Logic: a balancing act between keeping the **old** memory (h_{t-1}) vs. accepting the **new candidate** ($\tilde{h}_t$).

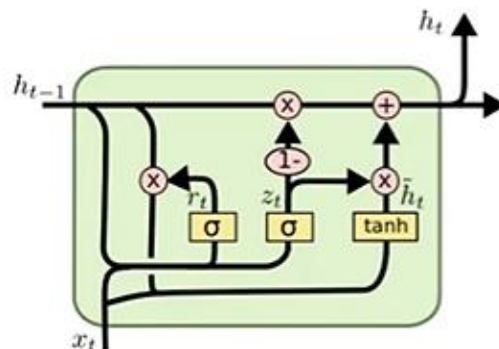
- Critical input (e.g. a plot twist) → more weight to the new candidate
- Filler/noise input (e.g. "the") → preserve the old memory instead

The cleanest one-line summary the article gives:

- **Reset Gate:** $h_{t-1} \rightarrow \tilde{h}_t$ (focus: **short-term** relevance)
- **Update Gate:** $\tilde{h}_t \rightarrow h_t$ (focus: **long-term** balance)

Tying this straight back to your King's Story example: at $t=3$ ("war, king killed"), the Reset Gate likely let a lot of the old "peaceful intro" context fade while drafting the new candidate — a real topic shift. At $t=4$ ("son grows strong"), the Update Gate had to balance "keep some Tragedy from before" against "let Power rise again" — exactly the kind of push-pull this phase describes.

GRU — Full Math Walkthrough (All 4 Steps)



Step 1 — Reset Gate (r_t)

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Values between 0-1: **near 0** = "shut the gate, ignore this part of the past"; **near 1** = "keep it."

Story example: King dies, son introduced. Past memory [Power=0.6, Conflict=0.6, Tragedy=0.7, Revenge=0.1] → Reset Gate [0.8, 0.2, 0.1, 0.9] → keep 80% of Power, **wipe** Conflict/Tragedy (fresh start for new character), but **keep 90%** of Revenge (that plot thread continues).

Step 2 — Candidate Hidden State ($\tilde{h}_t$)

First, apply the reset gate to the past (pointwise multiply):

$$h_{t-1}^{\text{reset}} = r_t \odot h_{t-1} = [0.8, 0.2, 0.1, 0.9] \odot [0.6, 0.6, 0.7, 0.1] = [0.48, 0.12, 0.07, 0.09]$$

Then mix with current input, pass through tanh:

$$\tilde{h}_t = \tanh(W_c \odot [r_t \odot h_{t-1}, x_t] + b_c)$$

Step 3 — Update Gate (z_t)

Same formula shape as reset gate, own weights:

$$z_t = \sigma(W_z \odot [h_{t-1}, x_t] + b_z)$$

Step 4 — Final Hidden State (h_t)

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Worked example, dimension by dimension:

- $z_t = 0.7$ (dim 1): $h_t = 0.3(0.6) + 0.7(0.7) = 0.18 + 0.49 = \mathbf{0.67}$ → shifted strongly toward new candidate
- $z_t = 0.2$ (dim 4): $h_t = 0.8(0.1) + 0.2(0.2) = 0.08 + 0.04 = \mathbf{0.12}$ → barely changed, mostly kept old memory

One clean rule: $z_t \rightarrow 1$ means "trust the new candidate," $z_t \rightarrow 0$ means "trust the old memory."

GRU vs LSTM — Full Comparison Table

	LSTM	GRU
Gates	3 (Input, Forget, Output)	2 (Reset, Update)
Memory	Cell state + Hidden state (separate)	Single Hidden state (merged)
Parameters	$\approx 4((d \odot h) + (h \odot h) + h)$	$\approx 3((d \odot h) + (h \odot h) + h)$ — 25% fewer
Speed	Slower	Faster
Performance	Slight edge on very complex, long-range tasks	Comparable, especially on smaller datasets
Best starting point	—	Often recommended default for new projects

This closes the loop on everything you've built: your GRU used hidden_size=64 for exactly this h — and beat LSTM on your IMDB task (87.22% vs 83.65%), which this article explicitly says can happen, especially on datasets of your scale. The parameter math also explains *why* GRU trained noticeably faster per epoch in your notebook, even though you didn't measure wall-clock time directly.

